```
%python
from databricks.feature_engineering import FeatureEngineeringClient
fe = FeatureEngineeringClient()

fe.set_feature_table_tag(name=f"{table_name}", key="Project", value="MLIA")
fe.set_feature_table_tag(name=f"{history_table_name}", key="Project", value="MLIA")
```

Figure 5.15 – Setting feature table tags on our table

Next, we define the case class structures. *Figure 5.16* shows that our case classes define our data structures. This is because Scala is a typed language.

```
case class InputRow(CustomerID: Long,
                    TransactionTimestamp: java.time.Instant)

case class TransactionCountState(latestTimestamp: java.time.Instant,
                                 currentTransactions: List[InputRow])

case class TransactionCount(CustomerID: Long,
                            transactionCount: Integer,
                            eventTimestamp: java.time.Instant,
                            isTimeout: Boolean)
```

Figure 5.16 – Defining class structures to support aggregations for each customer ID

In *Figure 5.17*, we see the `addNewRecords` function. `latestTimestamp` is calculated by comparing the latest timestamp in `transactionCountState` with the latest timestamp from the new records. This is in case we've received data out of order. Finally, we create and return the new `TransactionCountState` object with the newly calculated `latestTimestamp` and combine the two record lists:

```
def addNewRecords(newRecords: List[InputRow],
                  transactionCountState: TransactionCountState):
                  TransactionCountState =
 {
   val recordWithLatestTimestamp =
       newRecords.maxBy(record => record.TransactionTimestamp)
   val latestNewTimestamp = recordWithLatestTimestamp.TransactionTimestamp
   val latestTimestamp =
       if (latestNewTimestamp.toEpochMilli() > transactionCountState.
       latestTimestamp.toEpochMilli()
       ) latestNewTimestamp else transactionCountState.latestTimestamp

   new TransactionCountState(latestTimestamp,
       transactionCountState.currentTransactions ::: newRecords)
 }
```

Figure 5.17 – Defining the addNewRecords function